

Learning Word-Length-Specific Decision Trees for Hangman

An Empirical Study over 285,611 English Word Forms

Rareș Stana

Independent Portfolio Research Project

Abstract

This report investigates Hangman as a sequential decision-tree optimization problem. Instead of training a classifier to imitate another solver, HANGMANAI constructs a separate custom multiway decision tree for each word length. Internal nodes prescribe letter guesses, while child branches correspond to the exact positional pattern revealed by a guess. The vocabulary is derived from the complete English `wordfreq` “best” word list and cleaned to 285,611 usable ASCII word forms. The primary experiment treats every retained word as equally likely and evaluates each constructed tree against every word in its corresponding length bucket with six permitted mistakes. The strongest empirical pattern is a steep reduction in difficulty with increasing word length: current trees win only 4.00 % of three-letter games, 29.67 % of five-letter games, 67.63 % of ten-letter games, and 97.03 % of eleven-letter games, reaching 100 % from length twelve in the current experiments. The preferred first guess also changes by length, moving from **A** for lengths 3–6 to **E** for 7–11 and mostly **I** for 12–18. A secondary five-letter experiment shows that realistic word-frequency priors can change the learned root from **A** to **E** and substantially improve frequency-weighted performance. These results show that there is no single context-free answer to the familiar question of which letter should be guessed first in Hangman. However, the current large trees are best-found policies under greedy or bounded search budgets rather than proven global optima. The report therefore emphasizes both the empirical findings and the computational limits that shape them.

Keywords: decision trees, Hangman, sequential decision making, combinatorial optimization, word frequency, interpretable AI

1 Introduction

Hangman appears simple because its action space is small: at most 26 letters can be guessed, and each turn reveals whether a selected letter occurs in the hidden word. Strategically, however, the game combines uncertainty, limited mistakes, positional feedback, and a rapidly changing set of plausible words. A useful guess is therefore not necessarily the globally most common letter. Its value depends on the word length, the remaining candidate vocabulary, the positions already revealed, the letters already ruled out, and the number of mistakes still available.

The central question of this project is deliberately narrow:

If the Hangman policy itself must be represented as a decision tree, what tree gives the strongest strategy for each word length?

This framing differs from conventional supervised decision-tree learning. Classical methods such as ID3 induce a tree from labeled examples by repeatedly choosing useful feature tests (Quinlan, 1986). In HANGMANAI, the tree is itself the sequential policy. An internal node does not test a feature; it prescribes a letter to guess. The environment then returns one of several possible positional outcomes, and that observed outcome determines which child node is traversed next. The result is a custom multiway policy tree specialized to Hangman.

A second design choice is equally important: the project does not force every word length into a single model. A separate tree is constructed for three-letter words, four-letter words, five-letter words, and every other populated vocabulary bucket. This allows the experiments to directly test a widespread assumption that a single opening letter, commonly **E**, is universally best.

The project makes four contributions:

1. It formulates Hangman as a word-length-specific multiway decision-tree problem.
2. It builds and evaluates trees over a large corpus-derived English vocabulary containing 285,611 cleaned word forms.
3. It reports how win rate, mistakes, tree size, and preferred opening letter change with word length.
4. It compares uniform-vocabulary optimization with a frequency-weighted objective based on realistic English usage priors.

All numerical results reported in this paper are empirical outputs from the described experimental setup; no benchmark values are fabricated or manually adjusted.

2 Background and Research Questions

2.1 Decision trees as sequential policies

Decision trees are attractive for this problem because they are transparent: every action can, in principle, be traced to a node and every transition to an observed outcome. Interpretability is particularly useful in Hangman, where a user can inspect the exact sequence of guesses that led to a win or a loss.

The computational difficulty is that globally optimizing a decision tree is generally hard. Hyafil and Rivest showed that constructing optimal binary decision trees is NP-complete (Hyafil and Rivest, 1976). The Hangman setting is at least as structurally demanding in practice because nodes are multiway, branch probabilities depend on the current candidate set, and the objective includes a finite mistake budget. Consequently, the project distinguishes carefully between an *exact optimum* and the *best tree found* under a configured search budget.

2.2 Research questions

The current experiments target the following questions:

RQ1.

How does Hangman win rate change with hidden-word length under a length-specific decision-tree policy?

RQ2.

Does the preferred first guess change with word length?

RQ3.

Is difficulty explained primarily by the number of candidate words, or by the information provided by positional patterns?

RQ4.

How does the assumed probability distribution over English words affect the learned tree?

RQ5.

How much does the current computational budget constrain claims of optimality?

3 Vocabulary and Data Construction

3.1 Source vocabulary

The vocabulary is derived from the English “best” word list distributed with the `wordfreq` package. The package combines multiple frequency sources and provides both frequency estimates and an extensive ordered English lexicon, allowing the study to avoid an arbitrary top- N vocabulary cutoff (Speer, 2023).

Every token is lowercased and filtered to ASCII letters `a-z`. The current preprocessing additionally removes duplicates, words shorter than three characters, strings containing five or more repeated identical characters, and longer tokens without a vowel-like character. Each retained word is stored together with its length and its `wordfreq` Zipf frequency. The Zipf representation follows the broader use of logarithmic word-frequency scales motivated by psycholinguistic frequency research (Brysbaert and New, 2009).

3.2 Vocabulary distribution

After preprocessing, the dataset contains **285,611** usable word forms. The largest bucket contains 45,576 six-letter words. The distribution then gradually contracts as length increases.

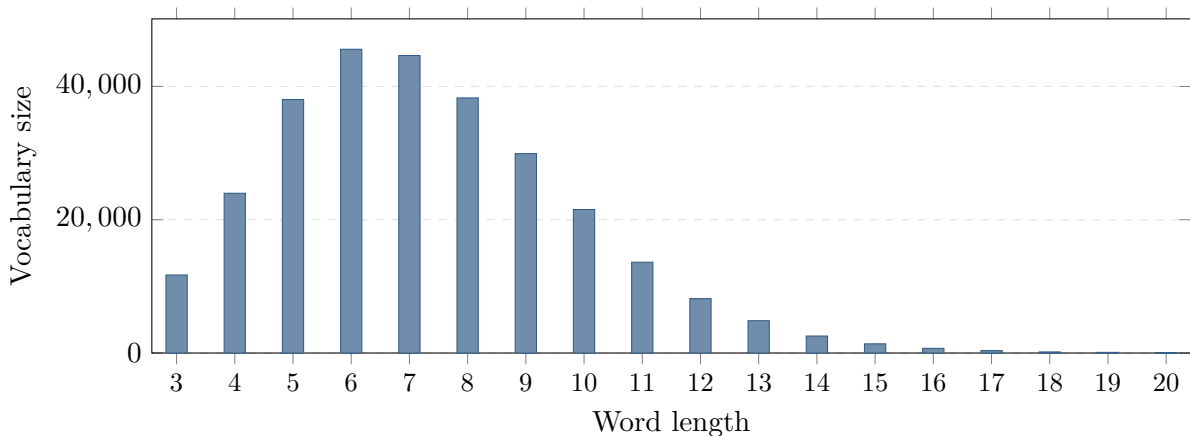


Figure 1: Number of cleaned `wordfreq` entries by word length for lengths 3–20. Very long buckets are omitted from the plot because they contain at most a few dozen entries.

The distribution matters because Hangman difficulty is jointly determined by the number of hypotheses and the informativeness of each revealed position. A shorter bucket can be harder than a larger bucket if its words remain difficult to distinguish after several guesses.

4 Hangman Decision-Tree Formulation

4.1 State and branching

For a fixed word length L , a Hangman state contains the visible pattern, guessed letters, incorrect letters, remaining lives, and the set of vocabulary entries still consistent with all observations. At a decision node, the tree prescribes an unguessed letter ℓ .

The outcome is a binary positional mask

$$o(w, \ell) \in \{0, 1\}^L, \quad (1)$$

where position i equals one if the hidden word w contains ℓ at position i . A five-letter guess can

therefore produce outcomes such as 00000, 01000, or 10001. Words producing the same mask enter the same child branch.

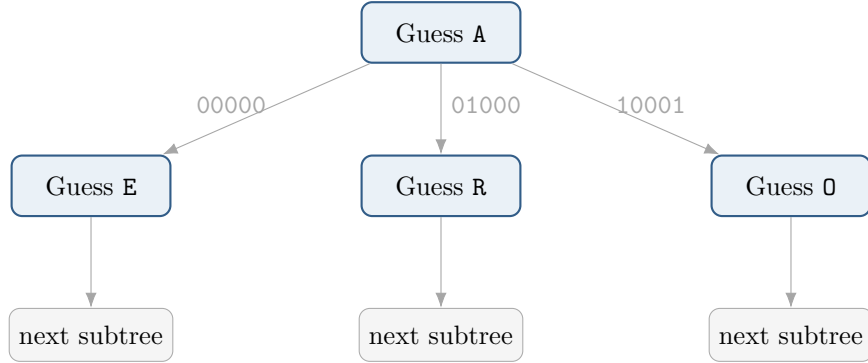


Figure 2: Conceptual structure of a multiway Hangman policy tree. Internal nodes prescribe a letter; branches represent exact reveal masks rather than simple yes/no outcomes.

4.2 Objective

The primary goal is to maximize the fraction of words solved before six incorrect guesses. For a tree T and vocabulary V_L of length L , the uniform objective is

$$J_{\text{win}}(T) = \frac{1}{|V_L|} \sum_{w \in V_L} \mathbb{I}[T \text{ solves } w]. \quad (2)$$

Ties are broken using lower average mistakes, fewer total guesses, smaller expected traversal depth, and smaller tree size. A secondary weighting mode replaces uniform mass with word-frequency-derived weights:

$$J_{\text{freq}}(T) = \frac{\sum_{w \in V_L} q(w) \mathbb{I}[T \text{ solves } w]}{\sum_{w \in V_L} q(w)}, \quad (3)$$

where $q(w)$ is obtained by converting the stored Zipf score into a positive linear weight.

5 Tree Construction and Experimental Setup

5.1 Construction strategies

Two construction regimes are used in the current project. Large vocabulary buckets primarily use a greedy split rule because exhaustive future-tree evaluation is computationally prohibitive. Smaller buckets use bounded lookahead, where candidate letters are evaluated using recursively estimated subtree statistics for a configurable depth. State estimates are memoized to avoid recomputing equivalent subproblems.

The automated profile is adaptive. Large buckets use a fast greedy configuration, while buckets with fewer words can receive deeper bounded lookahead. The full training command processes each length independently, making parallel execution and resume support straightforward.

5.2 Greedy scoring

The current greedy score prioritizes hit probability, then penalizes miss probability under life pressure, then prefers smaller expected child candidate sets and more distinct branches. This is computationally efficient but is not equivalent to globally maximizing eventual Hangman win rate.

Consequently, results from greedy buckets should be interpreted as best-found policies under the implemented heuristic.

5.3 Node budget and fallback behavior

Large trees are capped by a node budget. When the budget is reached, the present implementation stores a fixed fallback letter order based on the remaining candidate vocabulary. This makes the deployed policy compact, but it also means that a budget-capped “leaf” can still emit additional ordered guesses. Strictly speaking, those terminal actions are not all represented as explicit child nodes. This is an important limitation for a project whose idealized goal is a fully explicit decision tree, and it is discussed further in [section 8](#).

5.4 Evaluation protocol

Each trained tree is evaluated on **every retained word** of its corresponding length, not on a sampled subset. The headline metrics are:

- win rate;
- mean and median incorrect guesses;
- mean total guesses and remaining lives;
- maximum and average traversal depth;
- node and leaf counts;
- model file size, training time, and evaluation time.

All experiments use six allowed incorrect guesses unless otherwise specified.

6 Results

6.1 Overall performance by word length

[table 1](#) summarizes the principal results through length 20. The most striking pattern is monotonic improvement in win rate from lengths 3 through 12. Short words are exceptionally difficult despite having smaller vocabularies than some medium-length buckets.

Table 1: Current uniform-vocabulary results. “Greedy” and “bounded” refer to the construction strategy, not a claim of global optimality.

Len.	Words	First	Win %	Avg. wrong	Nodes	Strategy
3	11,707	A	4.00	5.90	1,099	greedy
4	23,971	A	17.55	5.61	9,999	greedy
5	38,039	A	29.67	5.34	20,058	greedy
6	45,576	A	31.19	5.29	20,085	greedy
7	44,632	E	35.10	5.08	20,156	greedy
8	38,276	E	40.89	4.75	20,187	greedy
9	29,916	E	51.36	4.10	20,228	greedy
10	21,531	E	67.63	3.00	20,257	greedy
11	13,624	E	97.03	0.96	20,085	greedy
12	8,161	I	100.00	0.55	12,373	greedy
13	4,846	I	100.00	0.55	6,535	bounded
14	2,549	I	100.00	0.45	3,404	bounded
15	1,378	I	100.00	0.39	1,813	bounded
16	700	I	100.00	0.28	915	bounded
17	368	I	100.00	0.33	482	bounded
18	164	I	100.00	0.22	198	bounded
19	97	R	100.00	0.19	118	bounded
20	42	A	100.00	0.10	52	bounded

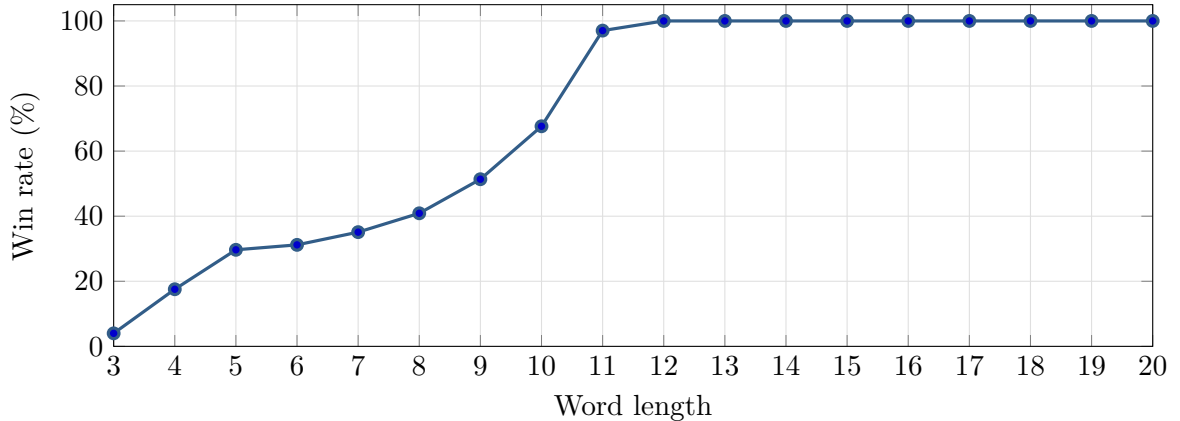


Figure 3: Full-vocabulary win rate by word length. The current policies improve sharply with length and reach complete coverage from length 12 onward in these experiments.

For lengths 3–11, the Pearson correlation between word length and win rate is $r = 0.949$, while the Spearman rank correlation is $\rho = 1.0$. These are descriptive summaries rather than significance tests, because each point is a deterministic full-bucket evaluation rather than an independent sample. Over the same range, average mistakes decline monotonically from 5.90 to 0.96 ($\rho = -1.0$).

6.2 Why short words are harder

The result is initially counterintuitive. A three-letter word contains fewer characters to reveal than an eleven-letter word, yet the three-letter tree performs far worse. The key is that Hangman provides *positional* information. A correct guess in a long word can reveal a distinctive pattern across many positions, sharply reducing the candidate set. A short pattern such as `_a_` may remain compatible with a large family of words.

Raw vocabulary size alone does not explain the observed difficulty. Across lengths 3–11,

vocabulary size has only a weak descriptive Pearson association with win rate ($r = -0.232$), and the corresponding Spearman correlation is approximately -0.05 . Six-letter words form the largest bucket, yet the six-letter policy is far stronger than the three- and four-letter policies. The evidence therefore supports the view that *candidate ambiguity after positional feedback* is more important than bucket size alone.

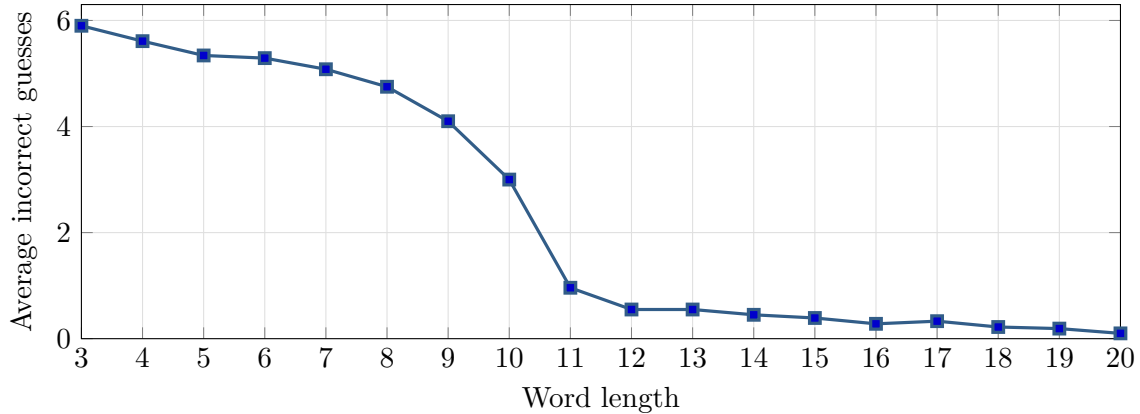


Figure 4: Average incorrect guesses decline strongly as word length increases.

6.3 The preferred first guess changes with length

The trained uniform trees exhibit three clear root regimes in the well-populated buckets. Lengths 3–6 begin with A; lengths 7–11 begin with E; and lengths 12–18 begin with I. Roots for the sparsely populated very-long buckets vary and should not be generalized.

This observation directly rejects the idea that a single opening letter is universally preferred under the current objective. Importantly, it does *not* yet prove that these letters are globally optimal roots: for the large buckets they are the roots selected by the current heuristic and node budget. Nevertheless, the fact that the preferred root changes systematically with length is a robust descriptive finding of the constructed policies.

6.4 Tree complexity

Tree size rises quickly for the ambiguous medium-length buckets. Lengths 5–11 cluster around the configured 20,000-node budget, indicating that these policies would continue expanding if given more capacity. By contrast, longer-word trees shrink rapidly because each branch resolves the vocabulary more efficiently and the candidate buckets themselves are smaller.

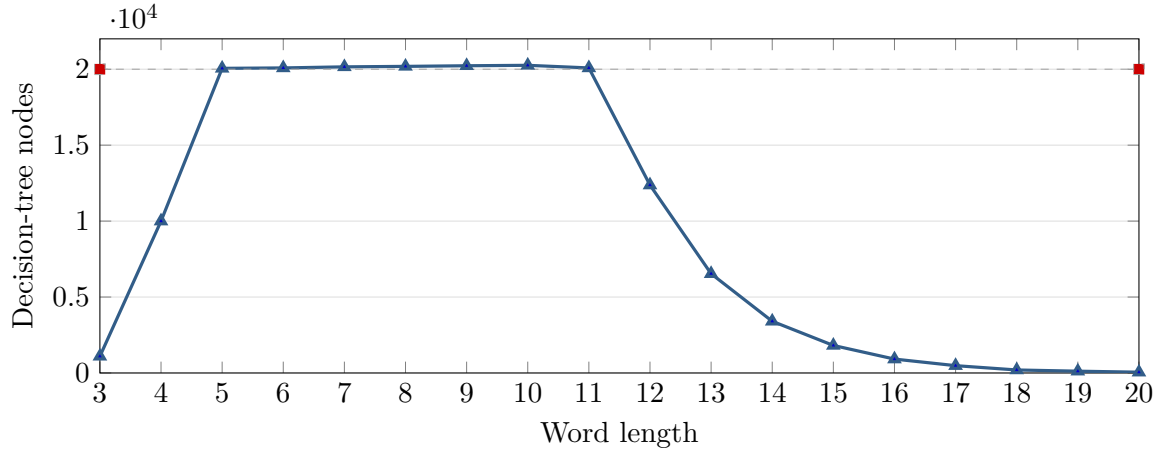


Figure 5: Tree size by word length. The dashed reference marks 20,000 nodes, near the budget reached by several large buckets.

6.5 Uniform versus frequency-weighted objectives

A five-letter secondary experiment tests whether a “best” strategy depends on how likely each hidden word is assumed to be. Under the uniform objective, every retained word has equal mass. Under the frequency-weighted objective, common words receive larger weights derived from `wordfreq`.

Table 2: Five-letter weighting experiment. Weighted metrics evaluate each tree under the `wordfreq`-derived prior.

Tree objective	Root	Unweighted win	Weighted win	Unw. wrong	Wtd. wrong	Nodes
Uniform	A	29.67%	34.92%	5.34	5.21	20,058
Wordfreq	E	27.38%	69.82%	5.41	3.73	20,048

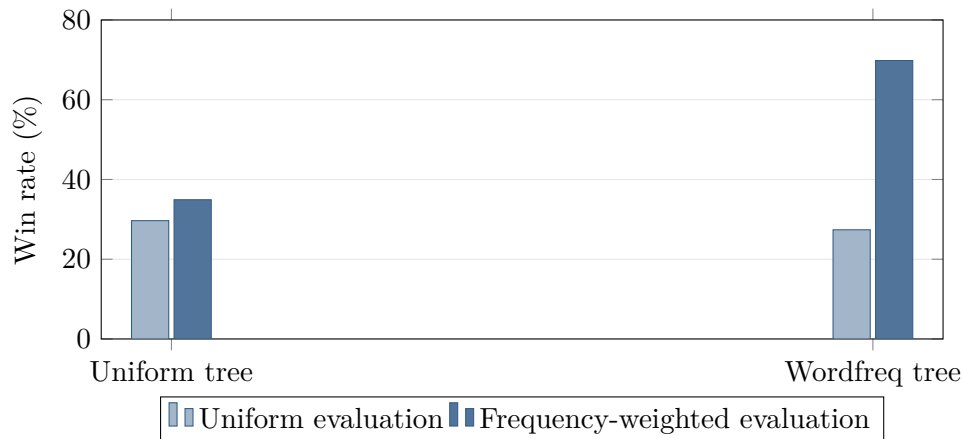


Figure 6: Five-letter trees under two evaluation distributions. Optimizing for common-word prior changes the root from A to E and greatly improves weighted coverage, while slightly reducing unweighted coverage.

The result is substantively important. Under uniform evaluation, the frequency-weighted tree is slightly worse, because it is willing to sacrifice rare entries. Under the realistic prior, however, weighted win rate rises from 34.92% for the uniform tree to 69.82% for the frequency-weighted tree.

The root simultaneously changes from A to E. Thus, “best Hangman strategy” is not well-defined without specifying the distribution from which hidden words are drawn.

6.6 Forced-root analysis

The experimental framework also supports forced-root sweeps in which all 26 letters are separately fixed as the first guess and the remaining subtree is constructed under the same root-analysis configuration. Initial sweeps have been completed for lengths 3, 4, and 5. The length-5 sweep prefers A, agreeing with the trained uniform tree. For length 4, the forced-root sweep ranks S above the trained tree’s A. For length 3, many roots tie on win rate and mistakes, with V leading only after later tie-breakers, whereas the trained greedy tree begins with A.

This disagreement is useful rather than problematic: it reveals sensitivity to search budget and construction procedure. The forced-root sweep currently uses a smaller default node budget than the primary training runs, so it should be interpreted as a diagnostic rather than a definitive proof of root optimality.

7 Discussion

7.1 RQ1: Word length is a dominant difficulty factor

The experiments provide strong descriptive evidence that short-word Hangman is substantially harder for the current decision-tree formulation. Between lengths 3 and 11, every increase in word length is accompanied by a higher win rate and a lower average mistake count. The transition is especially sharp between lengths 10 and 11, where win rate increases from 67.63% to 97.03%.

A plausible mechanism is the combinatorial richness of positional feedback. A hit in a long word can create a highly distinctive mask, whereas a hit in a short word often eliminates relatively few candidates. This makes longer words easier despite requiring more characters to be revealed. The result also explains why bucket size alone fails to predict difficulty.

7.2 RQ2: There is no universal first letter in the current policies

The first-guess progression from A to E to I is one of the clearest portfolio-level findings. It directly motivates the length-specific modeling choice. A universal strategy would hide this structure by forcing all lengths to share a common root decision.

At the same time, the distinction between a selected root and a proven optimal root is crucial. The current roots are properties of the implemented search procedure, and deeper optimization could alter them, especially for the short and medium buckets that hit node limits.

7.3 RQ3: Ambiguity matters more than vocabulary size alone

The weak association between vocabulary size and win rate supports a richer notion of Hangman difficulty. The relevant quantity is not simply how many words exist initially, but how efficiently letter-position outcomes partition those words. This suggests future analyses based on candidate-family structure, branch balance, positional-mask diversity, and the depth at which candidate sets become deterministic.

7.4 RQ4: Probability assumptions materially change the answer

The frequency-weighted five-letter experiment demonstrates a genuine objective trade-off. The uniform tree is optimized to cover rare and common tokens equally; the weighted tree focuses on words users are more likely to encounter. The weighted tree’s 69.82% prior-weighted win

rate, compared with 34.92% for the uniform tree under the same evaluation prior, shows that the hidden-word distribution is not a minor modeling detail. It changes both the root action and the meaning of performance.

This distinction is analogous to broader work on word-frequency norms: estimates that better reflect language exposure can improve the realism of downstream behavioral models (Brysbaert and New, 2009). In Hangman, the same principle determines which failures the optimizer considers costly.

8 Limitations

The current results are useful, but several limitations prevent a claim of globally optimal Hangman play.

Greedy construction for large buckets. The primary 3–12 letter trees are mostly built using a greedy rule. It prioritizes immediate hit probability before expected candidate reduction and does not evaluate all possible future subtrees. A different root could yield a worse immediate split but a stronger eventual policy.

Finite node budgets. Several trees reach approximately 20,000 nodes. Their performance is therefore capacity-limited. A learning curve over node budgets is required to determine whether win rate has converged.

Fallback actions at budget leaves. When the builder reaches its budget, the current implementation stores a ranked fallback letter sequence. Because these subsequent actions are not expanded into explicit branches, some inference behavior is better described as a tree plus a fixed terminal fallback policy. A future implementation should materialize the fallback sequence as explicit nodes if the goal is a literally pure tree at every action.

Vocabulary composition. `wordfreq` is corpus-derived rather than a curated classroom dictionary. Even after cleaning, it can retain names, slang, rare terminology, abbreviations that happen to be alphabetic, and cross-lingual tokens used in English text. The 11,707 retained three-letter forms are therefore much broader than what a human would normally consider a standard Hangman vocabulary. This makes short-word results especially severe.

Sparse long-word buckets. Results above approximately 18–20 letters have little general meaning because only dozens or single entries remain. A 100% win rate over a one-word bucket is trivial and should not be compared with 100% over thousands of twelve-letter words.

Optimality terminology. The report intentionally uses terms such as “best tree found” for greedy and bounded-search results. Exact optimality should only be claimed after exhaustive or otherwise certifying optimization. Given the known computational hardness of decision-tree optimization (Hyafil and Rivest, 1976), such proofs may only be practical for restricted vocabularies or aggressively reduced state spaces.

9 Future Work

The immediate research agenda follows naturally from the limitations.

9.1 Pure explicit-tree inference

Budget leaves should be replaced with explicit continuation nodes. Even if the fallback sequence remains frequency-derived during construction, every action and outcome should exist as a serialized branch. This would make the inference policy literally inspectable from root to terminal state.

9.2 Search-depth and node-budget curves

For each important length, the tree should be trained under increasing budgets, for example 5k, 10k, 20k, 50k, and 100k nodes, combined with lookahead depths of 1, 2, 3, and deeper where feasible. The key question is whether the root and win rate stabilize as search quality improves.

9.3 Near-exact optimization for short words

Three- and four-letter buckets are the most scientifically interesting because they are both difficult and structurally compact at the pattern level. Memoization, branch-and-bound, candidate-bitset representations, and tighter upper bounds on subtree win probability may permit substantially deeper optimization. A verified exact result on even a curated subset would provide a useful anchor for evaluating the greedy heuristic.

9.4 Curated common-English vocabulary

A second vocabulary should be derived by applying a frequency threshold or a curated lexical source. This would create two complementary tasks: *full corpus coverage* and *human-plausible Hangman*. Comparing roots and win rates across the two would quantify how unusual tokens affect strategy.

9.5 Full frequency-weighted study

The five-letter weighting result should be repeated for all major lengths. Of particular interest is whether the uniform root progression $A \rightarrow E \rightarrow I$ persists when common words dominate the objective.

9.6 Structural difficulty metrics

Future analysis should record candidate counts after each guess, branch entropy, positional-mask diversity, repeated-letter structure, and sizes of “word families” that differ in only one or two letters. These metrics could explain why individual words or lengths remain difficult without introducing a competing playing algorithm.

10 Conclusion

HANGMANAI reframes Hangman as an interpretable, word-length-specific decision-tree problem. Over 285,611 cleaned English word forms, the current experiments show that short-word Hangman is substantially harder than long-word Hangman, that the preferred opening letter changes with word length, and that realistic word-frequency priors can materially alter the learned policy.

The strongest empirical pattern is the rise in uniform win rate from 4.00% for three-letter words to 67.63% for ten-letter words and 97.03% for eleven-letter words, followed by complete coverage in the current experiments from length twelve. This improvement is not explained by vocabulary size alone; positional information appears to dominate raw candidate count. The second major finding is that there is no single preferred opening letter across the learned trees. Finally, the five-letter frequency experiment demonstrates that “best” depends on the assumed hidden-word distribution:

a uniform objective selects **A**, whereas a realistic frequency-weighted objective selects **E** and more than doubles weighted win rate.

These findings are already sufficient to reject a simplistic universal Hangman rule. The next step is not to add unrelated models, but to make the decision-tree search itself stronger and more rigorous. In that sense, the project has evolved from a game-playing demo into a compact study of sequential decision-tree optimization, search budgets, linguistic priors, and interpretable AI.

References

- J. Ross Quinlan. Induction of decision trees. *Machine Learning*, 1:81–106, 1986. doi: [10.1007/BF00116251](https://doi.org/10.1007/BF00116251).
- Laurent Hyafil and Ronald L. Rivest. Constructing optimal binary decision trees is NP-complete. *Information Processing Letters*, 5(1):15–17, 1976. doi: [10.1016/0020-0190\(76\)90095-8](https://doi.org/10.1016/0020-0190(76)90095-8).
- Marc Brysbaert and Boris New. Moving beyond Kucera and Francis: A critical evaluation of current word frequency norms and the introduction of a new and improved word frequency measure for American English. *Behavior Research Methods*, 41(4):977–990, 2009. doi: [10.3758/BRM.41.4.977](https://doi.org/10.3758/BRM.41.4.977).
- Robyn Speer. **wordfreq**: Access to word frequencies in over 40 languages. Python software package, version 3.1.1. <https://pypi.org/project/wordfreq/>.